Atelier Machine Learning

Analyse Comportementale Clientèle Retail

Module Machine Learning - GI2-

Atelier Pratique : E-commerce de Cadeaux

Préparé par Fadoua Drira

Document pédagogique — Chaîne complète de traitement

Exploration → Préparation → Modélisation → Évaluation → **Déploiement**

Année Univeristaire : 2025-2026

1 Configuration de l'environnement de travail

Ce projet doit être réalisé sous **VS Code** puis déposé (*push*) sur votre **GitHub** qui sera fourni pour évaluation.

1.1 Structure du projet

Créez l'arborescence suivante :

```
projet_ml_retail/
|-- data/                  # Base de donnees
|   |-- raw/              # Donnees brutes originales
|   |-- processed/        # Donnees nettoyees
|   \-- train_test/       # Donnees splittees (train/test)
|-- notebooks/             # Notebooks Jupyter (prototypage)
|-- src/                   # Scripts Python (production)
|   |-- preprocessing.py
|   |-- train_model.py
|   |-- predict.py
|   \-- utils.py
|-- models/                # Modeles sauvegardes (.pkl, .joblib)
|-- app/                   # Application web (Flask)
|-- reports/               # Rapports et visualisations
|-- requirements.txt       # Dependances (genere via pip freeze)
|-- README.md             # Documentation
\-- .gitignore
```

1.2 Environnement virtuel

```
# Creation de l'environnement virtuel
python -m venv venv

# Activation
# Windows:
venv\Scripts\activate
```

1.3 Fichier requirements.txt

Ce fichier liste toutes les dépendances nécessaires au projet. Il se génère automatiquement :

```
# Generer le fichier apres installation des packages
pip freeze > requirements.txt

# Utilite : permet a quiconque de reproduire exactement
# le meme environnement avec :
pip install -r requirements.txt
```

1.4 Fichier README.md

Un fichier `README.md` doit être créé contenant :

- Titre et description du projet
- Instructions d'installation (environnement virtuel, dépendances)
- Structure du projet expliquée
- Guide d'utilisation (comment exécuter les scripts)

2 Contexte & Mission

Vous êtes **data scientist** au sein d'une entreprise **e-commerce de cadeaux**. L'entreprise souhaite mieux comprendre sa clientèle pour :

— Personnaliser ses stratégies marketing

— Réduire le taux de départ des clients (*churn*)

— Optimiser son chiffre d'affaires

Vous disposez d'une **base de données complexe**, regroupant **52 features** issues de transactions réelles et de données complémentaires. Cette base est intentionnellement imparfaite pour vous permettre de maîtriser l'ensemble de la chaîne de traitement en data science.

3 Objectifs pédagogiques

Compétence	Description
Exploration	Analyser la qualité et la structure des données
Préparation	Nettoyer, encoder et normaliser les features
Transformation	Réduire la dimension via ACP
Modélisation	Appliquer clustering, classification et régression
Évaluation	Interpréter les résultats et proposer des recommandations
Déploiement	Créer une interface utilisateur avec Flask

4 Synthèse des features

4.1 Features numériques (1-17)

N°	Feature	Type	Description	Intervalle	Unité
1	CustomerID	Entier	Identifiant unique client	10000 - 99999	-
2	Recency	Entier	Jours écoulés depuis dernier achat	0 - 400	jours
3	Frequency	Entier	Nombre de commandes distinctes	1 - 50	cmd
4	MonetaryTotal	Float	Somme totale dépensée	-5000 - 15000	£
5	MonetaryAvg	Float	Montant moyen par commande	5 - 500	£
6	MonetaryStd	Float	Écart-type des dépenses	0 - 500	£
7	MonetaryMin	Float	Dépense minimale enregistrée	-5000 - 5000	£
8	MonetaryMax	Float	Dépense maximale enregistrée	0 - 10000	£
9	TotalQuantity	Entier	Quantité totale d'articles	-10000 - 100000	unités
10	AvgQtyPerTrans	Float	Moyenne articles par commande	1 - 1000	unités
11	MinQuantity	Entier	Quantité minimale commandée	-8000 - 0	unités
12	MaxQuantity	Entier	Quantité maximale commandée	1 - 8000	unités
13	CustomerTenure	Entier	Durée relation client	0 - 730	jours
14	FirstPurchase	Entier	Ancienneté premier achat	0 - 730	jours
15	PreferredDay	Entier	Jour semaine favori	0 (Lun) - 6 (Dim)	-
16	PreferredHour	Entier	Heure journée favorite	0 - 23	h
17	PreferredMonth	Entier	Mois année favori	1 - 12	-

4.2 Features numériques (18-34)

N°	Feature	Type	Description	Intervalle	Unité
18	WeekendRatio	Float	Proportion achats weekend	0.0 - 1.0	ratio
19	AvgDaysBetween	Float	Délai moyen entre commandes	0 - 365	jours
20	UniqueProducts	Entier	Nombre produits différents	1 - 1000	prod
21	UniqueDesc	Entier	Nombre descriptions uniques	1 - 1000	desc
22	AvgProdPerTrans	Float	Produits distincts par cmd	1 - 100	prod
23	UniqueCountries	Entier	Nombre pays associés	1 - 5	pays
24	NegQtyCount	Entier	Lignes quantité négative	0 - 100	occ
25	ZeroPriceCount	Entier	Lignes prix nul	0 - 50	occ
26	CancelledTrans	Entier	Transactions annulées	0 - 50	trans
27	ReturnRatio	Float	Taux retour (qty négatives)	0.0 - 1.0	ratio
28	TotalTrans	Entier	Total lignes transactionnelles	1 - 10000	lignes
29	UniqueInvoices	Entier	Factures distinctes	1 - 500	fact
30	AvgLinesPerInv	Float	Lignes par facture moyenne	1 - 100	lignes
31	Age	Float	Âge estimé client	18 - 81 ou **NaN**	ans
32	SupportTickets	Float	Tickets support ouverts	**-1**, 0-15, **999**	tickets
33	Satisfaction	Float	Note satisfaction	**-1**, **0**, 1-5, **99**	/5
34	Churn	Binaire	Indicateur départ client	**0** (fidèle), **1** (parti)	-

4.3 Features catégorielles (35-43)

N°	Feature	Card.	Description	Valeurs	Encodage
35	RFMSegment	4	Segment valeur client	Champions, Fidèles, Potentiels, Dormants	Ord/One-Hot
36	AgeCategory	7	Tranche âge client	18-24, 25-34, ..., 65+, Inconnu	Ordinal
37	SpendingCat	4	Niveau dépense	Low, Medium, High, VIP	Ordinal
38	CustomerType	5	Profil comportemental	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu	One-Hot
39	FavoriteSeason	4	Saison achat préférée	Hiver, Printemps, Été, Automne	One-Hot/Cycl
40	PreferredTime	5	Moment journée favori	Matin, Midi, Après-midi, Soir, Nuit	Ord/One-Hot
41	Region	8	Région géographique	UK, Europe (N/S/E/C), Asie, Autre	One-Hot
42	LoyaltyLevel	5	Niveau ancienneté	Nouveau, Jeune, Établi, Ancien, Inconnu	Ordinal
43	ChurnRisk	4	Niveau risque départ	Faible, Moyen, Élevé, Critique	Ordinal

4.4 Features catégorielles (44-52)

N°	Feature	Card.	Description	Valeurs	Encodage
44	WeekendPref	3	Préférence temporelle	Weekend, Semaine, Inconnu	One-Hot
45	BasketSize	4	Taille habituelle panier	Petit, Moyen, Grand, Inconnu	Ordinal
46	ProdDiversity	3	Diversité achats	Spécialisé, Modéré, Explorateur	One-Hot
47	Gender	3	Genre client	M, F, Unknown	One-Hot

Suite page suivante

Suite du tableau

N°	Feature	Card.	Description	Valeurs	Encodage
48	AccountStatus	4	État compte	Active, Suspended, Pending, Closed	One-Hot
49	Country	37+	Pays résidence	UK, France, Germany, etc.	Target Enc
50	Newsletter	1	Abonnement newsletter	**Yes** (unique)	**Suppr.**
51	RegistDate	Var.	Date inscription texte	"12/03/10", "2010-03-12"	Parsing
52	LastLoginIP	Unique	Adresse IP dernière connexion	"192.168.1.45"	Feature Eng.

5 Problèmes de qualité à résoudre

Type	Features	Fréq.	Traitement
Valeurs manquantes	Age	30%	Imputation (moyenne, médiane, KNN)
Valeurs aberrantes	SupportTickets, Satisfaction	5-8%	Détection et correction
Formats inconsistants	RegistrationDate	100%	Parsing et standardisation
Feature inutile	NewsletterSubscribed	100% cst	Suppression
Données brutes	LastLoginIP	100%	Extraction sous-features
Déséquilibre classes	AccountStatus, Churn	Var.	Rééquilibrage

6 Exploration et Préparation des Données

6.1 Exemples de fonctions à explorer

Voici les étapes clés à implémenter dans vos scripts Python (notamment dans `utils.py` et `preprocessing.py`) :

1. StandardScaler et Normalisation

StandardScaler : Centre les données (moyenne = 0) et réduit (écart-type = 1). Essentiel pour les algorithmes sensibles aux échelles (SVM, KNN, Régression logistique, Réseaux de neurones).

Note : Faire `fit_transform` sur tout le dataset avant le split = fuite de données (*data leakage*). La variable target (y) ne JAMAIS normaliser - C'est l'objectif a predire, pas une feature

2. Corrélation des Features et Multicolinéarité

Corrélation : Mesure la relation linéaire entre deux variables (-1 à 1). À explorer avec une heatmap.
Multicolinéarité : Quand plusieurs features sont fortement corrélées entre elles (ex : MonetaryTotal et MonetaryAvg). Problème pour la régression linéaire/logistique.
Solutions :

- Matrice de corrélation avec seuil (supprimer si |corrélation| > 0.8)
- VIF (Variance Inflation Factor) : VIF > 10 = multicolinéarité sévère
- Conserver celle ayant le plus de sens métier

3. Analyse en Composantes Principales (ACP)

Objectif : Réduire la dimension (52 features → 2-10 composantes) tout en conservant l'information.
Utilité :

- Visualisation en 2D/3D
- Réduction du bruit
- Accélération des calculs
- Éviter la malédiction de la dimensionnalité

Méthode : Décomposer la variance pour trouver les axes principaux d'inertie.

4. Feature Engineering

Création de nouvelles features à partir des existantes pour capturer plus d'information :

```
# Ratio depenses/recency
df['MonetaryPerDay'] = df['MonetaryTotal'] / (df['Recency'] + 1)

# Panier moyen
df['AvgBasketValue'] = df['MonetaryTotal'] / df['Frequency']

# Anciennete vs activite recente
df['TenureRatio'] = df['Recency'] / df['CustomerTenure']

# Extraction de features depuis RegistrationDate
df['RegYear'] = pd.to_datetime(df['RegistrationDate']).dt.year
df['RegMonth'] = pd.to_datetime(df['RegistrationDate']).dt.month
```

5. Suppression des Features Inutiles

Critères de suppression :

- Variance nulle (ex : NewsletterSubscribed = toujours "Yes")
- Trop de valeurs manquantes (>50%)
- Forte corrélation avec une autre feature (redondance)
- Importance nulle dans les modèles arborescents

6. Imputation des Valeurs Manquantes

Imputing = Remplacer les valeurs manquantes (NaN) par des valeurs substitutives.
Méthodes :

- **Moyenne/Médiane :** Pour données numériques symétriques/asymétriques
- **Mode :** Pour données catégorielles
- **KNN Imputer :** Remplace par la moyenne des k plus proches voisins
- **Iterative Imputer :** Régression pour prédire les valeurs manquantes
- **Valeur constante :** -999, "Inconnu" (pour marquer comme manquant)

Important : L'imputation se fait sur `X_train` puis on applique la même transformation sur `X_test` (comme pour le StandardScaler).

7. Parsing de Données

Parsing = Analyser et convertir des données d'un format brut vers un format structuré exploitable.

Exemple avec RegistrationDate :

```
import pandas as pd

# Probleme : formats inconsistants
# "12/03/10" (UK), "2010-03-12" (ISO), "03/12/2010" (US)

# Solution : parser avec to_datetime (infer le format)
df['RegistrationDate'] = pd.to_datetime(df['RegistrationDate'],
                                        dayfirst=True,  # Priorite UK
                                        errors='coerce')  # NaT si erreur

# Extraction de features
df['RegYear'] = df['RegistrationDate'].dt.year
df['RegMonth'] = df['RegistrationDate'].dt.month
df['RegDay'] = df['RegistrationDate'].dt.day
df['RegWeekday'] = df['RegistrationDate'].dt.weekday
```

Autres exemples de parsing :

- **LastLoginIP :** Extraire le pays via GeoIP, détecter si IP privée/publique
- **Texte :** Tokenization, extraction de mots-clés (descriptions produits)

7 Modélisation Avancée

7.1 Séparation Train/Test

Obligatoire pour évaluer la performance réelle du modèle sur des données jamais vues.

```
from sklearn.model_selection import train_test_split

# Separation 80% train - 20% test (stratifiee pour preserve la distribution)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,       # Reproductibilite
    stratify=y             # Conserver proportion classes (important pour Churn)
)

# Sauvegarde dans data/train_test/
X_train.to_csv('data/train_test/X_train.csv', index=False)
X_test.to_csv('data/train_test/X_test.csv', index=False)
y_train.to_csv('data/train_test/y_train.csv', index=False)
y_test.to_csv('data/train_test/y_test.csv', index=False)
```

7.2 Recherche des Meilleurs Hyperparamètres

GridSearchCV teste toutes les combinaisons possibles d'hyperparamètres et sélectionne la meilleure via validation croisée.

Optuna est plus intelligent que GridSearch : il utilise les résultats précédents pour guider la recherche (plus rapide, moins de combinaisons testées).

8 Déploiement : Interface avec Flask

8.1 Pourquoi Flask ?

Flask est un micro-framework Python léger et flexible, idéal pour :

— Créer des API REST pour servir vos modèles ML

— Développer rapidement une interface web simple

— Intégrer facilement avec des modèles scikit-learn

— Apprendre les bases du déploiement ML sans complexité excessive

Alternatives : FastAPI (plus rapide, async), Django (plus lourd, complet), Streamlit (très simple pour data apps).

Bon courage pour cet atelier pratique !

N'hesitez pas a consulter la documentation accessible sur le Web pour avancer.